

V | MANUAL

1. OBJETIVO DE VANTAGE · ID: MANUAL:OBJETIVO-001

El Problema que Resuelve

Una búsqueda laboral sin estructura produce cuatro fallas operativas concretas:

- Oportunidades de alta señal desaparecen antes de ser procesadas.
 - Tiempo consumido en vacantes irrelevantes que no cumplen criterios mínimos.
 - Aplicaciones enviadas sin datos de fit — sin score, sin análisis de keywords, sin estrategia de CV.
 - Sin trazabilidad: qué se aplicó, cuándo, qué sigue.

Qué Hace Diferente

VANTAGE convierte la búsqueda laboral en un pipeline con contratos de procesamiento definidos.

Filtra antes de evaluar: Links muertos → Score 0, Status Expirada. Roles sin componente visual → Gate BLOCKED. Empresas en lista negra → rechazadas en discovery. (La lista completa de empresas excluidas y la lógica de bloqueo vive en §10 — Gestión de Datos.)

Verifica antes de crear: Cada URL pasa un chequeo de enlace (lo que el sistema llama internamente "URL_GATE") antes de cualquier cálculo de fit. Si el link no funciona, la vacante no entra al pipeline activo. El mecanismo completo de este chequeo y los pasos que le siguen están explicados en §2 — Cómo Funciona, bajo "Gate Decisions".

Centraliza en un solo lugar: Notion es la fuente única de verdad — vacantes, aplicaciones, scores, seguimiento.

Calcula con lógica determinista: Score 0–100 calculado por Python. La decisión de postulación se toma con datos, no con estimaciones.

Impacto del Sistema (KPIs)

Evidencia de posicionamiento (Positioning Modes N1–N4) verificada — el detalle de qué es cada modo de posicionamiento se explica en §8.3 (Miércoles — CV Optimization), donde efectivamente se usan.

Lo que el Sistema No Hace

- No busca cualquier empleo — solo roles visuales en sectores lujo, premium, cool DNA y agencias de experiencia.
 - No genera volumen masivo — calidad de señal sobre cantidad de resultados.
 - No aplica automáticamente — la decisión de postulación es siempre humana.
 - No adivina campos faltantes — si falta información, el campo queda pendiente y el sistema lo reporta.

Para Quién Es Este Sistema

Perfil: Profesional senior (10+ años) en Visual Merchandising, Brand Environment, Store Design, Retail Experience.

Geografía: CDMX / LATAM.

Sectores target: Lujo (LVMH, Kering, Richemont), retail premium (Nike, Apple, Inditex), cool DNA (Gentle Monster, Ben & Frank), agencias de experiencia.

Las empresas excluidas permanentemente (Hard Blocks) y las reglas de deduplicación están documentadas de forma completa y única en §10 — Gestión de Datos. Este Manual evita repetir esa lista en más de un lugar para que nunca quede una copia desactualizada.

——2. CÓMO FUNCIONA · ID: MANUAL:FUNCIONAMIENTO-001

Flujo General del Pipeline

El pipeline opera secuencialmente. Cada paso tiene un responsable (una capa del sistema: L1, L2, L3, Python, o el operador humano) y un output definido antes de pasar al siguiente paso. El detalle operativo día por día de este flujo está en §8 — Flujo Semanal de Operación; aquí se explica la lógica que sostiene ese flujo.

División del Trabajo

El sistema se organiza en capas con responsabilidades separadas:

- L1 (Active Recon): búsqueda activa vía prompts ejecutados en motores de búsqueda (Perplexity, Comet).
 - L2 (Strategic Search): búsqueda activa complementaria vía Gemini, Grok, you.com.
 - L3 (Passive Intake): lectura automática de correos etiquetados en Gmail, tres veces al día, sin intervención humana.
 - L4 (Version Control & Infrastructure): mantiene sincronizados en background el repositorio de código (git) y los documentos fundacionales del sistema entre Notion y disco local.
 - Python (Pipeline / Runtime): normaliza, deduplica, calcula Score y Gate_Decision, y expone comandos de consulta.
 - El operador (tú): decide qué se postula, resuelve bloqueos recuperables, autoriza escrituras y aprueba entregables.
 - Este reparto de trabajo se ve en acción completa en §8, día por día.

Gate Decisions — cómo se decide dónde aterriza cada vacante

Este es uno de los conceptos que más se usa a lo largo de todo el ciclo operativo, así que conviene fijarlo aquí, antes de encontrarlo en Lunes, Martes o Miércoles sin previo aviso.

El sistema evalúa cada vacante nueva en tres pasos, siempre en este orden:

1. Link check — si la URL de la vacante no carga (404, 403, dominio caído, redirección rota), la vacante se archiva automáticamente con Score 0 y Status "Archivar". No se calcula nada más sobre ella: un link muerto no tiene fit que evaluar. Esto es lo que el sistema llama internamente el "URL_GATE" — el primer filtro que cualquier vacante debe pasar antes de que Python invierta cómputo en analizarla.
2. Score (0–100) — si la URL funciona, Python calcula un puntaje numérico según qué tan bien encaja el rol con el perfil objetivo (keywords de Visual Merchandising, sector, seniority, geografía). Este cálculo es determinista:

mismos datos de entrada, mismo score de salida, siempre.

3. Dónde aterriza, según el Score obtenido:

- 60 o más → Ready-to-Apply (tu bandeja de trabajo diaria — de aquí sale todo lo que trabajas el Miércoles en CV Optimization).
 - 40–59 → Para Revisar (zona gris: el sistema no descarta la vacante, pero tampoco la prioriza — la decisión de trabajarla o no es tuya).
 - Menos de 40 → Archivar (descartada; ver §3 — Filosofía de Fallo, para entender por qué esto no es un error que debas corregir).
 - Excepción a los tres pasos: si la vacante llegó por contacto directo (Inbound, Referencia o Networking), se salta este proceso completo y entra directo como CREATE — un contacto humano pesa más que el algoritmo, porque la señal de calidad ya viene validada por una persona real, no por texto de un JD.
 - Este mismo mecanismo de Gate es el que determina si una vacante queda en estado BLOCKED cuando algo en sus datos de entrada (Class A: URL, JD, Source_Type, Prioridad) es inconsistente — ese caso específico y cómo recuperarlo se cubre en detalle en §8.2 (Martes — Dashboard).

Lógica de Filtrado: Soft vs Hard Blocks

El sistema aplica dos capas de exclusión para garantizar la calidad de la señal — ambas se explican con su lista completa y mecánica de recuperación en §10 — Gestión de Datos, pero conviene entender la diferencia conceptual desde ahora, porque ambos términos aparecen constantemente en el flujo semanal:

- Hard Blocks (Permanentes): empresas o roles que nunca entrarán al sistema (ej. L'Oréal, Levi's). Se filtran en el origen, antes de que la vacante exista siquiera como registro en Notion, y no son recuperables bajo ninguna circunstancia.
 - Soft Blocks (Contextuales): vacantes bloqueadas por inconsistencias en datos Class A (URL rota, JD parcial) o por score insuficiente. A diferencia de los Hard Blocks, estas sí son recuperables — corrigiendo el dato erróneo a través del Dashboard (ver §8.2, Martes).

——3. FILOSOFÍA DE FALLO PARA OPERADORES · ID:

MANUAL:FALLO-001Base: KERNEL:FAIL-PHILOSOPHY.

Antes de entrar a Setup y al ciclo operativo, es necesario internalizar esto, porque vas a encontrarlo constantemente desde el primer Lunes que operes el sistema:

Un “fallo” del sistema no es un bug — es el filtro operando correctamente. Los siguientes resultados son señales normales de funcionamiento, no errores que debas reparar manualmente ni forzar a que Claude “arregle”:

Resultado que verás	Qué significa realmente	Qué NO hacer	Qué hacer en su lugar
---------------------	-------------------------	--------------	-----------------------

URL dead / link roto	La vacante expiró — es normal del mercado laboral, las publicaciones caducan.	No repares manualmente el link ni intentes “revivir” la vacante.	Déjala archivada. Si te parece un error de captura (ej. la URL se guardó mal, no que la vacante haya expirado), eso sí es corregible — ver §8.2, Dashboard.
Score = 0	El fit es débil, o el link estaba muerto (ver §2, Gate Decisions, paso 1).	No subas el score a mano — es un campo Class B, calculado por Python, no editable directamente.	Si sospechas que el cálculo está mal por un dato de entrada erróneo (URL, JD), corrige el dato de entrada, no el resultado — ver §8.2.
Gate = BLOCKED	Los criterios Class A no se cumplieron (URL rota, JD parcial, fuente no reconocida).	No lo fuerces a CREATE manualmente en Notion.	Si es recuperable, usa el Dashboard (§8.2, Martes) para corregir el dato de origen y dejar que el pipeline recalcule.
Ready-to-Apply vacío	No hay oportunidades válidas esta semana — puede pasar, especialmente en semanas de baja actividad del mercado.	No fuerces un CREATE artificial para “llenar” la bandeja.	Espera al siguiente ciclo de discovery (Lunes), o revisa si el Prompt de búsqueda necesita ajuste (ver §12, Health Check — Red Flags).

JSON vacío en el Feed de discovery	La búsqueda no encontró resultados relevantes esa ejecución.	No amplíes los criterios de búsqueda sin análisis previo — podrías bajar la calidad de la señal general.	Revisa el Viernes de Analytics (§8.5) antes de decidir si el prompt necesita ajuste.
Ante cualquiera de estos casos, el sistema reporta el estado y espera tu instrucción dentro del flujo normal del pipeline — no requiere, ni acepta bien, intervenciones manuales que intenten "corregir" el resultado en sí mismo en vez de corregir el dato de entrada que lo produjo.			

——4. SETUP · ID: MANUAL:SETUP-001

Esta sección se ejecuta una sola vez, al instalar el sistema por primera vez (o al reinstalarlo desde cero). Si el sistema ya está instalado y solo llevas varios días sin usarlo, lo que necesitas es §5 — Arranque Frío, no este capítulo completo.

Prerrequisitos

- Cuenta de Notion con base de datos VANTAGE TRACKER activa.
 - Python 3.8+ instalado en Mac.
 - Acceso a Claude.
 - Cuenta de Perplexity con modo Deep Research activo.
 - Acceso a Gemini con modo Deep Research o Search activo.
 - Acceso a You.com con modo Research o Agent activo.
 - Acceso a Grok con modo DeepSearch o Think activo.

Paso 1 — Verificar Notion

Abre VANTAGE TRACKER y confirma que existen estas cuatro vistas:

- Ready-to-Apply — espacio de trabajo diario (Score ≥ 60).

- Para Revisar — vacantes en rango Score 40–59.
- Archivar — Score 0 o Status Expirada.
- All — administración general.
- Si VANTAGE TRACKER no existe, configúralo antes de continuar — el resto del setup asume que estas cuatro vistas ya existen.

Paso 2 — Instalar Entorno Python

```
cd ~/Documents/03 Projects/VANTAGE/Layer_1
source .venv/bin/activate
# (el entorno ya existe; solo actívalo)
```

Verifica la instalación: python3 --version debe mostrar 3.8 o superior.

Paso 3 — Configuración de Claude (Bootstrap)

Ya no es necesario realizar copy-paste manual del System Prompt maestro en cada actualización.

1. Copia el STATIC BOOTLOADER v1.0 (disponible en la Cédula Digital) y pégalo en Settings → Project → Project Instructions en la UI de Claude.
2. Inicia un nuevo chat. El Agente Vantage realizará un fetch automático de la gobernanza activa desde Notion.
3. Confirma que el Agente responde con “VANTAGE v8.9.4: SISTEMA SINCRONIZADO” antes de enviar peticiones.
4. Nota: este setup de Claude es de una sola vez por proyecto — no se repite en cada sesión de trabajo. Lo que sí se repite en cada sesión es el Ciclo de Sesión completo, explicado en §6.

Paso 4 — Verificar Archivos del Sistema y Permisos de Ejecución

Confirma que los archivos del sistema existen en tu Mac en las rutas esperadas (Layer_1, Layer_3, Layer_4, Dashboard). Si reinstalas o mueves archivos, verifica permisos de ejecución:

```
chmod +x $LAYER_1_DIR/layer_1_pipeline.sh
chmod +x $LAYER_1_DIR/wrappers/layer_1_wrapper.sh
chmod +x $LAYER_3_DIR/wrappers/layer_3_mail.sh
chmod +x $DASHBOARD_DIR/wrappers/dashboard_start.sh
```

Paso 5 — Test Inicial del Pipeline

```
~/vantage_pipeline.sh tracker
```

Output esperado:

```
= VANTAGE PIPELINE STATUS =
Ready-to-Apply: [N] vacantes
Para Revisar: [N] vacantes
...
```

Si falla: verifica que ~/vantage_notion_audit/.env existe y contiene tu token de Notion.

Paso 6 — Verificar VANTAGE Runtime

El Runtime (explicado en detalle en §9) es el motor de lectura del sistema — permite consultar el estado de Notion desde terminal sin abrir el navegador. Antes de operar, verifica que su índice interno (el "Entity Index", el catálogo de todas las entidades que el Runtime sabe interpretar) esté cargado:

```
python vantage.py status
```

Resultado esperado: Status: READY (4,200+ blocks indexed).

Paso 7 — Verificar Sync Documental (vsync_doc)

```
cd ~/Documents/03 Projects/VANTAGE/Layer_4/scripts
source ../../Layer_1/.venv/bin/activate
python vsync_doc.py --dry-run
```

Output esperado: 6 documentos listados con diff por documento, sin errores.

Si falla: verificar que layer_1.env exista y que el token no tenga un salto de línea (\n) embebido por error de copy-paste.

5. ARRANQUE FRÍO — Checklist de Reactivación

Usar cuando el sistema no ha sido operado por más de 5 días. A diferencia de Setup (§4), aquí no estás instalando nada nuevo — estás confirmando que todo lo que ya instalaste sigue funcionando después de un periodo de inactividad, antes de confiar en que el primer comando que corras te va a dar un resultado correcto.

1. Verificar entorno

```
cd ~/Documents/03 Projects/VANTAGE/Layer_1/scripts
source ../.venv/bin/activate
python3 --version # debe ser 3.8+
```

2. Verificar token Notion

```
cat ../.env | grep NOTION_TOKEN # debe estar presente y
no expirado
# Si expirado: regenerar en Notion → Settings → API → New
token
```

3. Status del Runtime

```
python3 vantage.py status
# Revisar: total_entities, hash_coverage,
index_age_hours, warning
```

4. Sincronizar Entity Index

```
python3 vantage.py sync
# Esperar: status: "ok", entities_after >=
entities_before
```

5. Verificar Tracker en Notion

Abrir VANTAGE TRACKER → vista "All"
Buscar entradas con Status = REVIEW_NEEDED
Resolver cada una: corregir campo → Status: Target →
correr pipeline

6. Verificar L3 (layer_3_mail.py)

Correr vl3 manualmente una vez
Verificar heartbeat: `cat ~/.vantage/l3_heartbeat.json`
Si falla: verificar layer_3.env (IMAP credentials,
GROQ_API_KEY)

7. Smoke test final

`python3 vantage.py ask "show active roles"`
`python3 vantage.py ask "find candidates"`
Confirmar que la lista refleja el estado actual de
Notion

Una vez que este checklist pasa limpio, el sistema está en condición de operar con normalidad — el siguiente paso es abrir tu primera sesión de trabajo del día, lo cual te lleva directo a §6.

6. CICLO DE SESIÓN — Open/Close · ID: MANUAL:SESSION-CYCLE-001

Cuándo se dispara esto, y por qué es distinto del ciclo semanal

El ciclo semanal que se detalla en §8 (Lunes → Viernes) asume que el sistema documental y el estado del pipeline están sanos al momento de empezar a trabajar. Esa suposición no es gratuita: cada vez que abres una conversación nueva con Claude para operar VANTAGE, esa conversación pasa primero por su propio ciclo de vida — independiente del ciclo semanal, y que existe precisamente para que nunca operes sobre un supuesto sin verificar.

Piensa en esto como el equivalente a revisar que las luces del tablero no tengan ninguna advertencia antes de arrancar el coche: no es el viaje en sí, es la condición para que el viaje no te sorprenda a medio camino.

Este ciclo se dispara con dos comandos — `/vantage-session-open` al inicio de cada sesión, `/vantage-session-close` al final — y hace tres cosas que ningún otro punto del sistema hace:

1. Deja un registro de que la sesión existió y en qué estado terminó (el Session Ledger).
2. Confirma que los 6 documentos fundacionales + el Census están todos en la misma versión (nunca uno adelantado y otro atrasado).
3. Te recuerda, sin que tengas que preguntarlo, qué quedó pendiente de la

sesión anterior.

4. No necesitas invocarlo tú manualmente cada vez que se te ocurra — pero sí necesitas recordar que es el primer paso obligatorio: si acabas de abrir Claude para trabajar en VANTAGE hoy, el primer paso siempre es este ciclo, antes de tocar Tracker, Dashboard o cualquier trigger de CV descrito en §8.

Por qué existe esto

Antes de que este ciclo existiera, cada sesión de Claude arrancaba “en frío”: el agente asumía que el corpus de Notion (Kernel, Manual, System Prompt, Career Canon, Aliases, Changelog) estaba en la versión que recordaba de la sesión anterior, y no había ningún mecanismo que confirmara si una sesión había terminado bien o si Claude simplemente dejó de responder a medio trabajo — un timeout, un cierre accidental de pestaña, un crash. El resultado era drift silencioso: un documento se actualizaba, otro no, y nadie se enteraba hasta que las contradicciones aparecían en producción (esto es, en parte, lo que motivó el Census — ver KERNEL:CENSUS-SYNC, y §11 — Health Check, donde se detalla cómo se regenera el Census).

El ciclo Open/Close resuelve esto tratando cada sesión como una transacción con inicio y fin explícitos, en vez de una conversación que simplemente “pasa”.

Un cambio de fondo: el operador corre Terminal primero, Claude ya no improvisa la verificación

Antes de entrar al detalle paso a paso, vale la pena explicar un giro importante en cómo funciona este ciclo desde la optimización de `verify_versions.py`: el script dejó de ser una herramienta de un solo modo (verificar versión) y ahora opera con tres flags — `--bootstrap`, `--check` y `--sync` — cada uno con una responsabilidad distinta y acotada. Este cambio no es cosmético: reordena quién hace qué entre tú y Claude.

En el diseño anterior, Claude tenía margen para decidir cómo resolver la verificación de versión si Terminal no estaba a la mano — podía hacer un fetch directo vía MCP a cada uno de los 7 documentos fundacionales, más lento pero funcional, como downgrade aceptable. Ese margen desaparece por completo en el flujo actual. Terminal ya no es la ruta preferente — es un requisito obligatorio. Si Terminal no está disponible, la sesión simplemente no avanza: no hay fallback, no hay downgrade de eficiencia, no hay pregunta de “¿puedes correr el script?”. Claude ya no pregunta porque Claude ya no tiene la opción de resolverlo por otra vía.

La razón detrás de esta decisión es doble: primero, reduce el margen de error de que Claude intente reconstruir el estado del sistema leyendo bases de datos completas vía MCP — una operación cara en tokens y con más superficie para malinterpretar un dato a medias. Segundo, mantiene bajo el conteo de tokens de cada sesión, porque los tres outputs que generan los flags de `verify_versions.py` son bloques de texto compactos, ya resueltos, que Claude simplemente consume — no bases de datos completas que Claude tiene que parsear e interpretar por su cuenta.

Esto convierte el ciclo en un modelo asimétrico y determinista: tú corres Terminal primero y le entregas a Claude los resultados ya calculados; Claude nunca reconstruye ese cálculo por su cuenta, ni lo intenta.

Apertura — /vantage-session-open

Con ese cambio de fondo entendido, así es como se ve la apertura en la práctica, paso a paso:

- 1. Corre `python3 scripts/verify_versions.py --bootstrap` en tu Terminal local. Este es el flag nuevo, y su función reemplaza algo que antes hacía Claude por su cuenta: leer la última fila del Session Ledger para confirmar que cerró bien, y leer la última entrada de V-CHANGELOG para tener contexto de continuidad. Ahora ambas lecturas ocurren en Terminal, no en Claude — el script te entrega un bloque de texto delimitado por `[DUMP INICIO SESIÓN VANTAGE]`, que contiene exactamente esa información: el estado de la última sesión (si cerró en CLOSED o quedó OPEN por una terminación abrupta), el resumen de la última entrada del Changelog, y un snapshot de los tickets CRÍTICO y ALTO pendientes en el Bug/Task Tracker.
- 2. Corre `python3 scripts/verify_versions.py --check` en la misma Terminal. Este flag es el que ya conocías — el que responde a la pregunta “¿están todos los documentos hablando de la misma versión del sistema, o alguno quedó atrás?”. Es de solo lectura: itera los 7 `page_id` fijos (Kernel, Manual, Career Canon, System Prompt, Aliases, Changelog, Census) y trae únicamente la propiedad Versión de cada uno, sin tocar el contenido de los bloques. El output se ve así:

```
=
VANTAGE – MANIFEST CHECK (read-only)
=
[v] V | ALIASES          X.X.X          [OK]
[v] V | SYSTEM PROMPT   X.X.X          [OK]
[v] V | KERNEL          X.X.X          [OK]
[v] V | MANUAL          X.X.X          [OK]
[v] V | SESSION LEDGER  X.X.X          [OK]
[v] V | ID CENSUS       X.X.X          [OK]
-----
PASS – all components at X.X.X
```

- 1. Abre un chat nuevo en Claude, pega ambos outputs — el dump del `--bootstrap` y la tabla del `--check` — e invoca `/vantage-session-open`. Claude ya no hace ningún fetch propio a Notion en este paso: los dos payloads que le entregaste son toda la información que necesita para arrancar. Si alguno de los dos outputs falta o está incompleto, Claude te lo señala y espera a que lo completes antes de continuar — no improvisa el faltante ni lo reconstruye por su cuenta vía MCP.

2. Claude procesa ambos payloads y realiza una sola escritura: crea la fila nueva en el Session Ledger (KERNEL:SESSION-LEDGER) con status: OPEN y un session_id que genera él mismo. Esta escritura es de housekeeping — no requiere APROBAR_WRITE, por la misma razón que tampoco lo requiere el auto-sync del Entity Index (KERNEL:HEALTH-CHECK-001): no toca campos Class A ni Class B del Tracker, es infraestructura de continuidad de sesión, no dato de negocio del pipeline de vacantes.
3. Claude confirma el estado de la sesión. Si la tabla del --check mostró PASS limpio en los 7 documentos, responde VANTAGE: SISTEMA SINCRONIZADO y queda listo para recibir instrucciones normales de trabajo. Si detecta un drift de versión en el output que le pegaste, lo reporta explícitamente antes de continuar — recuerda que este es exactamente el mismo tratamiento que ya conocías: el drift no bloquea el trabajo salvo que el documento desincronizado sea justo el que vas a editar en esa sesión, en cuyo caso primero se resuelve el drift.
4. Al terminar estos cinco pasos, recién ahí es seguro empezar con el Lunes de §8, o cualquier otro día del ciclo semanal que corresponda.

Cierre — /vantage-session-close

El cierre es la contraparte simétrica, y conserva la misma lógica de fondo que la apertura: primero corres Terminal, después le entregas a Claude los resultados ya resueltos para que cierre el ciclo sin tener que reconstruir nada por su cuenta.

1. Si algún ID cambió de estado durante la sesión (de pendiente a resuelto, o se creó uno nuevo), corre generate_census.py en Terminal — el mismo comando que ya conocías, que regenera el Census y de paso te señala cualquier ID huérfano detectado en los documentos fuente. Este paso sigue siendo obligatorio antes de escribir el Changelog, nunca al revés: si el Census no puede regenerarse en este momento porque no tienes Terminal a la mano, el ticket asociado se marca Blocked-Census en vez de darse por cerrado en falso — eso no cambió.
2. Corre python3 scripts/verify_versions.py --check de nuevo en Terminal, esta vez para confirmar que no quedó ningún desfase de versión entre los documentos antes de cerrar. Copia el PASS que te entrega.
3. Corre python3 scripts/verify_versions.py --sync en Terminal. Este es el segundo flag nuevo, y es el único de los tres que escribe: toma la versión target definida en tu entrada de Changelog de esta sesión, la lee vía pages.retrieve, y ejecuta seis peticiones pages.patch secuenciales — una por documento fundacional restante — actualizando directamente la propiedad "Versión" de cada página en Notion, sin pasar por Claude. Igual que --bootstrap reemplazó una lectura que antes hacía Claude, --sync reemplaza una escritura que antes ocurría como parte del paso de "version bump" del cierre — ahora ocurre en Terminal, de forma local,

antes de que Claude confirme nada. Es housekeeping puro, exento de APROBAR_WRITE por la misma razón que el resto de las escrituras de infraestructura descritas arriba: APROBAR_WRITE queda reservado exclusivamente para modificaciones de contenido, código o IDs en los documentos fundacionales — nunca para la sincronización mecánica de un número de versión ya decidido. Copia el SYNC COMPLETE que te entrega el script al terminar.

4. Pega ambos outputs — el PASS del paso 2 y el SYNC COMPLETE del paso 3 — en el chat, e invoca /vantage-session-close. Claude valida que el gate de sincronización esté limpio con lo que le entregaste, y a partir de ahí ejecuta lo que ya hacía antes: presenta el inventario de cambios de la sesión (qué se tocó — documentos editados, entradas de Tracker modificadas, IDs creados o resueltos), te entrega el resumen automático de hecho/pendiente sin que lo pidas, y cierra la fila del Ledger que quedó abierta al inicio de la sesión: status: OPEN pasa a status: CLOSED, con el resumen de pendientes escrito en pending_summary — el mismo campo que la próxima sesión leerá en el dump de su propio --bootstrap.
5. La sesión termina con el mensaje SESIÓN COMPLETADA → nuevo chat, que sigue siendo tu señal de que es seguro cerrar la ventana o empezar una conversación nueva sin perder continuidad.

Qué hacer si algo no cuadra

- Terminal no está disponible → Operación detenida (Fail-Fast). No existe bypass automatizado vía MCP para este chequeo — ni en apertura ni en cierre — con el fin de proteger la cuota de la API de Notion y evitar el desperdicio de tokens de contexto reconstruyendo estado desde bases de datos completas. El operador debe diagnosticar la conectividad o el entorno local antes de proceder con cualquier modificación de documentación.
 - El Ledger anterior quedó OPEN → esto lo verás reflejado directamente en el dump que genera --bootstrap. Revisa manualmente si algo quedó a medio escribir antes de seguir. El sistema te lo señala, pero la decisión de qué hacer con eso es tuya.
 - Drift de versión detectado y no es el documento que ibas a tocar hoy → se reporta, no bloquea. Puedes decidir resolverlo ahora o después.
 - Drift de versión detectado y SÍ es el documento que ibas a tocar → se resuelve el drift primero, antes de aplicar cualquier parche nuevo — de lo contrario terminarías escribiendo sobre una base que ya no coincide con lo que las otras piezas del sistema esperan.
 - Con la sesión abierta y sincronizada, el siguiente paso natural es abrir tu mapa de la semana — el V-Checklist, explicado en §7.

——7. EL CHECKLIST Y LAS INTERFACES COMPARTIDAS · ID: MANUAL:VCHECKLIST-001

El Checklist — tu acompañante durante todo el ciclo

El V-Checklist (V-CHECKLIST · Vantage Weekly) es la interfaz operativa de todo el ciclo semanal que se detalla en §8. Es un archivo HTML autocontenido con progreso persistente (localStorage), modo claro/oscuro y navegación por día. Ábrelo una vez al iniciar la semana y consúltalo a lo largo de las actividades de cada día — no es una herramienta puntual de un solo momento, es tu mapa de avance de lunes a viernes.

Ubicación: archivo local Checklist.html (abre en navegador).

Reset: botón “🔄 Reset semana” en el header para iniciar un nuevo ciclo.

Persistencia: el progreso NO persiste entre sesiones distintas del navegador si se limpia el localStorage.

Dónde viven los archivos compartidos

Dashboard.html, Checklist.html, vantage-tokens.css y vantage-theme.js viven todos en Dashboard/ (no en HTMLs/ — esa carpeta fue un intento paralelo descartado, archivado en Dashboard/archive/ si decides conservarlo como evidencia).

- vantage-tokens.css define colores y superficies compartidos por ambos HTML.
 - vantage-theme.js es el toggle de tema compartido, con persistencia y sincronía cross-tab.
 - Esto importa porque el Dashboard (que verás en detalle en §8.2, Martes) usa exactamente esta misma infraestructura visual — no son dos sistemas de interfaz distintos, son la misma base compartida.

Tema claro/oscuro

El botón de tema (ícono sol/luna, arriba a la derecha en ambos HTML) persiste tu elección y se sincroniza automáticamente si tienes ambos HTML abiertos en pestañas distintas del mismo navegador — cambias el tema en uno, el otro se actualiza sin recargar. Antes del parche de 2026-07-10, el tema no se guardaba y siempre reiniciaba en oscuro.

Qué NO hacer

No copies/pegues código de un HTML al otro para “igualar” un color o componente — edita vantage-tokens.css o vantage-theme.js, que ambos ya leen. Editar directo en el HTML reintroduce el mismo drift que se corrigió.

Si algo se ve distinto entre los dos HTML, es señal de que alguien editó un color o estilo directo en el <style> inline de uno de los dos archivos en vez de en vantage-tokens.css. Revisa ahí primero.

Con el Checklist abierto y el ciclo de sesión ya confirmado (§6), estás listo para empezar el Lunes — el primer día del ciclo semanal, detallado a continuación.

8. FLUJO SEMANAL DE OPERACIÓN · ID: MANUAL:FLUJO-001

Este es el ciclo completo de trabajo, de lunes a viernes. Asume que ya pasaste por Setup (§4) o Arranque Frío (§5) según corresponda, que la sesión actual de Claude ya pasó por su Ciclo de Sesión (§6), y que tienes el Checklist (§7) abierto como guía de avance.

8.1 LUNES — Búsqueda Activa Completa (Discovery)

El lunes es el ciclo de búsqueda activa completo. Se dispara manualmente y cubre las dos capas de búsqueda humana (L1 y L2), más la revisión de lo que L3 recolectó de forma pasiva durante la semana.

El ciclo comienza con los prompts de búsqueda, los cuales no se copian de versiones anteriores — se ensamblan bajo demanda a través de Perplexity Desktop: cada prompt combina dos capas: el Prompt Base (perfil, reglas de exclusión, etc.) + el Prompt Wrapper (que contiene la fecha del día TODAY'S DATE, el modo de búsqueda, etc.).

Abre Perplexity Desktop y dale la instrucción:

Lee PROMPT LIBRARY vía MCP. <https://dub.sh/app.notion>

Recupera:

- BASE SPEC L1
- Wrapper Career Sites
- Wrapper LinkedIn
- Wrapper Aggregators

Concatena cada wrapper con BASE SPEC.

Sustituye TODAY'S DATE por la fecha actual.

Entrega los tres prompts completos listos para ejecutar.

No expliques el proceso.

No describas la arquitectura.

Entrega únicamente los prompts finales.

Recupera:

- BASE SPEC L2
- Wrapper Gemini
- Wrapper Grok
- Wrapper you.com

Concatena cada wrapper con BASE SPEC.

Sustituye TODAY'S DATE por la fecha actual.

Entrega los tres prompts completos listos para ejecutar.

No expliques el proceso.

No describas la arquitectura.

Entrega únicamente los prompts finales.

Puedes pedir uno o varios en el mismo mensaje; Perplexity lee Prompt Bases y

Prompt Wrappers desde PROMPT LIBRARY vía MCP a Notion, los concatena en un bloque de instrucciones y fecha automáticamente, y devuelve un bloque por wrapper listo para ejecutar. (La biblioteca completa de prompts y wrappers disponibles está catalogada en §13 — Prompts & Wrappers.)

Por qué importa la fecha: TODAY'S DATE define la ventana de búsqueda activa (14 días preferente, hasta 21 con match fuerte). Un prompt con fecha incorrecta produce resultados fuera de ventana o advertencias innecesarias en todos los ítems.

Nota sobre Score 0: si el chequeo de URL (ver §2 — Gate Decisions, paso 1) detecta un link roto o inaccesible, la vacante recibe automáticamente Score 0 y Status "Archivar". Esto es comportamiento esperado, no un error — ver §3, Filosofía de Fallo.

Los Prompts de L1 siempre necesitarán ser concatenados usando el Prompt de alguno de los siguientes Wrappers: Career Sites (Tier 1 → LVMH, Kering, Richemont; Tier 2 → Nike, Apple, Inditex; Tier 3 → cool DNA, agencias de experiencia), LinkedIn o Job Boards (OCC, CompuTrabajo, Indeed, Bumeran).

Algunas de las opciones con las que solicitarlos a Perplexity:

"Entrégame los prompts de L1"

"Entrégame los prompts de Career Sites"

"Entrégame el prompt de LinkedIn"

"Entrégame el prompt de Aggregators"

En Comet Desktop, usando Perplexity con el control del navegador activado, ejecutarás cada bloque en una pestaña diferente. Cada ejecución produce un JSON independiente. Compila los JSONs; los usarás en el paso de consolidación más abajo.

Regresa a Perplexity Desktop y solicita en esta ocasión los prompts de L2: Prompt A, Prompt B o Prompt C.

El Prompt A siempre necesitará ser concatenado con alguno de los siguientes Wrappers: Gemini, Grok o you.com. Los Prompts B y C pueden ejecutarse de manera independiente. Algunas de las opciones con las que solicitarlos a Perplexity:

"Entrégame el prompt de Gemini"

"Entrégame el prompt de Grok"

"Entrégame el prompt de you.com"

"Entrégame el prompt B"

"Entrégame el prompt C"

Ejecutarás cada bloque de instrucciones en su motor de búsqueda correspondiente usando Deep Research siempre que te sea posible. Los Prompts B y C pueden ser utilizados en cualquiera de los tres motores de búsqueda. Cada ejecución produce un JSON independiente. Compila los JSONs; los usarás en el siguiente paso.

En preparación para entrar al Pipeline es necesario consolidar la información

recopilada. Regresarás a Perplexity Desktop y, usando como base el Prompt E, pegarás los JSONs de L1 + L2.

Perplexity aplicará dedup con clave compuesta brand+title+location siguiendo una jerarquía L1 > L2: de las vacantes duplicadas persistirán las instancias de L1, tomando de L2 la información que pueda complementar sus propiedades para Class A.

Perplexity entregará como respuesta un Plain Array consolidado (JSON plano sin capas anidadas), listo para Python. Guardarás el resultado en:

```
~/Documents/03 Projects/VANTAGE/Layer_1/Feeds/YYYY-MM-DD_consolidated.json
```

Nota importante: vantage_merge.py está deprecado. El flujo correcto es: JSONs L1+L2 → Perplexity (Prompt E) → Plain Array → feed_processor.py. La jerarquía de dedup L1 > L2 descrita arriba es la misma lógica que rige la deduplicación general del sistema, detallada con más contexto en §10 — Gestión de Datos.

L3 lee los correos no leídos de Gmail que tengan asignado el label .Jobs., extrae vacantes con Groq y las escribe directamente en el Tracker. El ciclo es autónomo y corre en background vía launchd tres veces al día (08:00 · 14:00 · 21:00) — no requiere que tú hagas nada para que esto ocurra.

Ejecutarás manualmente solo si el equipo estuvo apagado en los horarios indicados o necesitas procesar backlog de Gmail antes del siguiente ciclo automático. Ejecutarás la app LAYER 3.app desde /Applications o usando Terminal:
v13

Para este momento, los siguientes filtros ya habrán sido aplicados sin consumir cuota: Hard-blocked (L'Oréal · Levi's/Dockers · El Palacio de Hierro — ver lista completa en §10), asuntos de agradecimiento, newsletters, confirmaciones de cuenta.

Límites por ejecución: procesa máximo 5 correos por run (configurable en GROQ_MAX_EMAILS_PER_RUN). Si hay backlog, el script reporta cuántos quedan.

Si L3 falla: verifica que LAYER_3/config/layer_3.env existe y contiene las credenciales de Gmail, Groq y Notion. El venv hereda de LAYER_1/.venv — si Layer 1 funciona, L3 tiene el entorno listo. (Para troubleshooting detallado de L3, ver §12.)

Abre la Terminal y procesa el JSON consolidado de L1+L2:

```
~/vantage_pipeline.sh feed ~/Documents/03 Projects/VANTAGE/Feeds/YYYY-MM-DD_consolidated.json
```

¿Qué ocurre aquí? El script vantage_pipeline.sh actúa como wrapper: activa el entorno virtual (.venv), valida la estructura y dispara feed_processor.py para normalizar campos, aplicar dedup cross-layer (ventana 30 días — ver §10) y presentarte el DRY RUN antes de escribir en Notion.

APROBAR ESCRITURA: revisa el DRY RUN en terminal. El output muestra las

propiedades Class A de cada instancia a crear. Las entradas duplicadas aparecen como SKIP. Las que requieren revisión aparecen como REVIEW_NEEDED. Confirma con y (yes) para escribir en Notion. Cualquier otra tecla cancela sin escribir. Los registros con status REVIEW_NEEDED que se escriben en Notion se resuelven al día siguiente en el Dashboard (§8.2, Martes).

PROCESAR CON PYTHON: para este punto las propiedades Class A de cada instancia nueva se habrán poblado por L1, L2 o L3. Para poblar las propiedades Class B de todas las instancias pendientes en el Tracker, ejecutarás la app LAYER 1.app desde /Applications o usando Terminal:

```
~/vantage_pipeline.sh
```

READY-TO-APPLY: abre la vista Ready-to-Apply en Notion. Vacantes con Score ≥ 60 están listas para CV Optimization en preparación para tu postulación — esto es lo que trabajarás el Miércoles (§8.3).

L4 mantiene dos cosas sincronizadas en background, sin intervención manual en el ciclo semanal normal: el repositorio git del sistema (vgit) y los 6 documentos fundacionales entre Notion y el disco local (vdoc). Son dos herramientas separadas que se combinan: vdoc mueve contenido documental Notion $\leftrightarrow$ ACTIVE/, y al terminar dispara automáticamente un git_sync — por eso casi nunca necesitas correr vgit a mano después de un vdoc.

vgit — Git Auto-Sync

Automático: launchd corre vgit a las 09:00 · 15:00 · 21:00. Si hay cambios sin commitear en el repo, hace commit con timestamp + push a origin/main. Sin cambios $\rightarrow$ silencio total, no notifica nada.

Manual: ejecutar vgit desde Terminal en cualquier momento para forzar un sync inmediato — útil si hiciste cambios locales fuera del ciclo automático y no quieres esperar al siguiente horario.

Verificar último run: `cat /tmp/vantage_l4_gitsync.log` — cada corrida (automática o manual) queda registrada ahí con timestamp, exit code y el output completo del sync, así puedes auditar qué pasó sin depender de las notificaciones del sistema. Si el repo no existe o está corrupto, vgit ya no lo confunde con “sin cambios” — reporta el error explícitamente y la notificación del wrapper se ve roja (✗), no verde.

Archivos: `Layer_4/scripts/git_sync.py` · `Layer_4/wrappers/git_sync_wrapper.sh` · `~/Library/LaunchAgents/com.vantage.gitsync.plist`

vdoc — Document Layer Sync

Sincroniza los 6 documentos fundacionales (Kernel · System Prompt · Career Canon · Manual · Aliases · Change Log) entre Notion y ACTIVE/ en disco, y al terminar encadena un git_sync automático para que el commit quede reflejado en GitHub sin un paso adicional.

Tres direcciones posibles:

- vdoc auto — compara la fecha de modificación de cada documento (local vs. Notion) y sincroniza en el sentido que corresponda, documento por documento. Es el modo por defecto y el más seguro para uso diario:

nunca sobrescribe algo más reciente con algo más viejo.

- vdoc notion — fuerza Notion → local para los 6 documentos, sin comparar fechas. Úsalo solo si sabes que Notion tiene la versión correcta y quieres descartar cualquier cambio local.
- vdoc local — fuerza local → Notion para los 6 documentos, sin comparar fechas. Úsalo solo si editaste los .md directamente en disco (offline) y quieres que Notion adopte esa versión.
- Como notion y local sobrescriben sin comparar fechas, ambos son operaciones forzadas: antes de ejecutar nada, vdoc te muestra automáticamente un preview (equivalente a --dry-run) de lo que va a hacer, y te pide confirmación explícita en terminal (s para continuar, cualquier otra tecla cancela). Si por alguna razón corres el comando sin una terminal interactiva disponible, el script no asume que confirmaste — cancela por seguridad y no escribe nada. vdoc auto nunca pide esta confirmación porque nunca sobrescribe algo más reciente.
- Modificador dry — se combina con cualquiera de los tres comandos anteriores y con cualquier documento específico, en cualquier orden, y siempre gana: nunca escribe en Notion, en disco ni hace commit, sin importar qué más hayas escrito en la misma línea.
- vdoc dry — preview de auto (equivalente a vdoc auto dry)
- vdoc notion dry — preview de lo que haría vdoc notion, sin ejecutar la escritura forzada
- vdoc local dry — preview de lo que haría vdoc local
- vdoc kernel dry — preview de solo Kernel en modo auto
- Recomendación operativa: corre siempre la variante dry primero cuando no estés seguro de qué dirección va a ganar — te cuesta segundos y evita sorpresas, especialmente antes de un notion o local forzado.
- Sync quirúrgico por documento — cualquiera de los 6 nombres puede pasarse solo o combinado con dirección/dry: vdoc kernel · vdoc system_prompt · vdoc career_canon · vdoc manual · vdoc aliases · vdoc change_log. Sin dirección explícita, cada uno corre en modo auto (gana el más reciente) solo para ese documento — los otros 5 no se tocan. Se puede combinar con notion/local (ej. vdoc notion kernel fuerza solo Kernel Notion→local) y con dry (ej. vdoc kernel dry).
- Archivos: Layer_4/scripts/vsync_doc.py (motor de sync) · Layer_4/scripts/vdoc.py (wrapper de comandos, el que invocas por alias) · reutiliza .venv de Layer_1.
- Ver también §12 — Troubleshooting, entrada "vsync_doc.py falla con error blocks.children.list() returned None".

——**8.2 MARTES — Recuperación y Dashboard · ID:**

MANUAL:DASHBOARD-001

Antes de avanzar al miércoles, este es el momento de resolver lo que quedó

bloqueado el lunes: REVIEW_NEEDED · BLOCKED recuperables · NADs vencidas. Las vacantes que recuperes aquí son las que estarán disponibles en Ready-to-Apply para trabajar mañana.

Si el bloqueo es por un campo Class A corregible, usa el Dashboard: Proponer Patch → Validar → Aceptar. No uses el Dashboard para forzar un CREATE en vacantes que no cumplen score — úsalo solo para corregir datos erróneos.

(Recuerda de §2: un Gate BLOCKED no es un error del sistema a "saltarse", es una vacante cuyos datos de entrada tienen un problema identificable y corregible.)

Es una sola herramienta (dashboard.html + dashboard_server.py :8000), no hay pestañas ni vistas separadas. La pantalla es un panel de recuperación de vacantes bloqueadas, con una tira de estado del pipeline (L1 → RT-1 → Notion → Mail) como indicador visual — no una vista de navegación distinta. Comparte la infraestructura visual (vantage-tokens.css, vantage-theme.js) con el Checklist, descrita en §7.

Archivos: Dashboard/dashboard.html · Dashboard/scripts/dashboard_server.py.

Abrir el Dashboard: ejecuta en terminal:

vd

El wrapper dashboard_start.sh arranca el servidor Flask en <http://127.0.0.1:8000> (accesible también vía Tailscale desde otros dispositivos), ejecuta un smoke test automático y abre dashboard.html en el navegador. Output esperado en terminal: SMOKE PASSED — abriendo dashboard. Si el smoke falla, emite notificación sonora de error (Basso) y no abre la UI. El indicador "BACKEND OK/OFFLINE" en la esquina superior confirma la conexión en vivo.

Partes del Dashboard:

- Sidebar (columna izquierda): estado de la instancia activa — instance_id, payload actual de la vacante (campos Class A como aparecen en Notion), capabilities disponibles en el estado actual (can_patch · can_validate · can_accept · can_archive) y Audit Log en tiempo real con cada evento registrado.
 - Panel principal (área derecha): cuatro secciones — selector de vacante (dropdown con todas las vacantes en Gate = BLOCKED, botón Crear instancia), máquina de estados FSM (visualiza el estado actual: BLOCKED → PATCHED → RETURNED_TO_CREATE), panel de patch (formulario con campos Class A editables: URL · JD · Source_Type · Prioridad), y área de resultado de validación (PASS verde o FAIL rojo con motivo).
 - Botones: Crear instancia · Proponer Patch · Validar · Aceptar Patch · Archivar · Sincronizar.
 - Secuencia — vacante BLOCKED recuperable:
- 1. Selecciona la vacante del dropdown (muestra Marca · Rol · Score · VM_Scope).
- 2. Crear instancia — abre una instancia en estado BLOCKED y carga el payload desde Notion. Audit Log registra domain.instance.created.
- 3. Edita los campos incorrectos en el panel de patch — solo Class A (URL ·

- JD · Source_Type · Prioridad). Los campos Class B no son editables.
4. Proponer Patch — almacena la corrección. Audit Log registra `domain.patch.proposed`.
 5. Validar — el backend ejecuta `run_pipeline.py` con el patch y verifica si el resultado sería CREATE. Si pasa: estado → PATCHED, resultado verde. Si falla: estado permanece BLOCKED, resultado rojo con motivo.
 6. Aceptar Patch — escribe los campos Class A corregidos en Notion. Estado → RETURNED_TO_CREATE. Audit Log registra `domain.patch.accepted`.
 7. Corre el pipeline para que Python recalcule:
`~/vantage_pipeline.sh`

Secuencia — vacante no recuperable: usa el botón Archivar. El Dashboard escribe `Next_Action = Archivar` en Notion y cierra la instancia en estado FAILED. No pasa por el pipeline.

Las entradas con este status son escritas en Notion por `feed_processor.py` cuando no pudieron procesarse completamente: la URL era parcial o ambigua, la marca no resolvía contra el alias map, o el sistema detectó un semi-duplicate cross-layer que requiere revisión humana. Mientras el status permanezca en REVIEW_NEEDED, sus campos Class B (Score, Gate_Decision, VM_Scope, Role_Class) quedan bloqueados — Python no los calcula.

Contrato de resolución — 4 pasos obligatorios:

1. Abre la entrada en Notion e identifica el problema indicado en el campo Notas (ej. "URL parcial", "alias no resuelto: Nike México", "semi-duplicate").
2. Corrige el campo problemático directamente en Notion: reemplaza la URL parcial con la URL completa, o ajusta el nombre de la marca al valor que exista en el alias map.
3. Cambia Status → Target. Este es el único valor que Python reconoce como señal de resolución. Cualquier otro valor (incluyendo dejar REVIEW_NEEDED) mantiene el bloqueo en el siguiente run.
4. Corre el pipeline:
`~/vantage_pipeline.sh`

Python detecta Status = Target en entradas que tenían Gate vacío o REVIEW_NEEDED y procesa sus campos Class B normalmente — calcula Score, Gate_Decision y el resto.

Por qué Target y no otro valor: Target es el estado operativo estándar de una vacante en espera de procesamiento. Usarlo como señal de resolución mantiene el contrato de estados consistente — no requiere un valor nuevo ni lógica adicional en Python.

Nota importante: estas entradas no pasan por el Dashboard. El

Dashboard es para vacantes con Gate = BLOCKED que ya tienen campos Class B calculados y necesitan corrección de inputs Class A. REVIEW_NEEDED es un estado previo — todavía no llegó a tener Gate calculado.

8.3 MIÉRCOLES — CV Optimization

Optimización de CV para vacantes priorizadas en Ready-to-Apply. Claude opera activamente en este ciclo — es el único día donde el AI Component tiene rol principal. L3 sigue corriendo en sus horarios habituales, en background.

Abre Ready-to-Apply en Notion y elige la vacante a trabajar. Copia la URL del campo URL (career page oficial) o el texto del JD. Abre una nueva sesión de Claude (recuerda: esto significa pasar primero por el Ciclo de Sesión de §6 si aún no lo has hecho hoy) y dispara:

CV-A [URL de la vacante]

o pega el texto del JD directamente. Claude no accede al Tracker de forma autónoma — el trigger debe ser explícito.

CV-A es análisis: qué keywords posicionar, qué gaps cubrir, qué tono de marca adoptar. CV-B es producción: el documento final. En una sesión única, el contexto de análisis contamina la voz del CV. La separación es una restricción de calidad, no de conveniencia.

Claude extrae los 6 keywords de posicionamiento del JD, identifica los gaps entre los requisitos del rol y el perfil de experiencia canónico del Career Canon, determina el Positioning Mode aplicable — hay cuatro posibles, definidos en el Career Canon:

- N1 Luxury Brand Execution
 - N2 Store Design & Flagship
 - N3 Regional Brand Execution
 - N4 Commercial VM & Field Leadership
 - Además, define el tono de marca del CV y detecta el idioma del JD (ES/EN) para el output.
 - Output de la sesión — el HANDOFF, 6 campos obligatorios:

```
{  
  "empresa": "",  
  "rol": "",  
  "JD_keywords_top6": ["", "", "", "", "", ""],  
  "fit_gaps": ["", ""],  
  "tono_marca": "",  
  "idioma": ""  
}
```

La sesión termina aquí. No se escribe ningún CV en CV-A.

HANDOFF — Contrato de Transferencia: CV-B no inicia con un HANDOFF

incompleto. Si cualquier campo está ausente, el sistema lo solicita antes de continuar.

PROTOCOL UPDATE — SKELETON-FIRST: CV-B ya no tiene permiso creativo sobre la estructura. El proceso es de inyección en slots: se usa el Golden Skeleton como base, y se vacía la información del Career Canon en los slots existentes sin alterar sus IDs.

Abre una sesión nueva de Claude. Pega el HANDOFF completo y dispara:
CV-B [pega el HANDOFF]

Claude verifica los 6 campos, cruza el HANDOFF contra el contrato de output del Career Canon para validar que bullets y KPIs sean derivados canónicos (no inventados), aplica el Positioning Mode definido en CV-A, usa el campo idioma del HANDOFF para seleccionar la versión ES o EN de cada sección del Career Canon (no se generan CVs bilingües ni se mezclan idiomas dentro de un mismo output) y genera el CV bajo ese mismo contrato de output.

El output tiene tres partes obligatorias y secuenciales:

1. Markdown con Figma tags — Claude entrega el archivo .md completo en la misma sesión. Cada slot va encabezado por su tag (##### figma_text_id). El operador lo revisa y autoriza antes de cualquier escritura en Notion.
 2. Autorización explícita del operador — Claude espera confirmación antes de continuar. Sin autorización, no escribe nada.
 3. Documentar la URL del Markdown.
 4. Regla de orden: el Markdown nunca se escribe en Notion si el operador no ha autorizado explícitamente. El orden cronológico de experiencia es invariante: C01 → C02 → C03 → C04 → C05. No se reordena por vacante ni por Positioning Mode.
 5. Escritura en Notion (dos destinos):
- Página en DERIVED OUTPUTS · ARCHIVE del Career Canon — con footer de Positioning Mode activo y fecha.
 - Bloque # MARKDOWN CANON ALIGNED en la página de la vacante en el Tracker — el Markdown completo con Figma tags, dentro de un bloque de código markdown.
 - Con el .md autorizado en mano, el flujo hacia Figma es directo — el plugin hace el trabajo pesado.
 - Instalación del plugin (una sola vez, si aún no lo tienes instalado): Figma Desktop → Plugins → Development → Import plugin from manifest... → navega a ~/Documents/03 Projects/VANTAGE/Figma Sync/ → selecciona manifest.json. El plugin queda disponible permanentemente. Es importante saber que el plugin no modifica Notion ni el Tracker — opera exclusivamente sobre el lienzo Figma activo.
 - Uso operativo, cada Miércoles:

1. Abre Figma Desktop y el archivo del CV.
 2. Plugins → Development → VANTAGE CV Sync.
 3. Copia el contenido completo del .md de CV-B y pégalo en el área de texto del plugin.
 4. Haz clic en Inyectar a Nodos Nativos.
 5. Verifica la notificación: VANTAGE Sync: X nodos actualizados vía Registry V2 (ID crudo).
 6. Revisa el lienzo visualmente y exporta: frame del CV → Export → PDF.
 7. Si el plugin reporta Keys sin resolver, revisa la entrada correspondiente en §12 — Troubleshooting ("Figma plugin no resuelve IDs").
- QA [adjunta el PDF exportado]

Claude revisa formato y completitud con checklist de 6 ítems y entrega go/no-go.
QA no evalúa fit — evalúa que el documento esté correcto como entregable.
Si QA aprueba, cambia Status a Postulado en Notion y corre:
`~/vantage_pipeline.sh`

Python detecta el Status y asigna Gate_Decision = APPLIED. La vacante sale de Ready-to-Apply automáticamente.

8.4 JUEVES — Segunda Pasada (Condicional)

Ejecuta solo si hay nuevas vacantes que procesar — 10 minutos máximo:
`~/vantage_pipeline.sh`

Script: `~/vantage_pipeline.sh`. Este día no tiene un procedimiento distinto al ya descrito en el Lunes (§8.1) — es simplemente una repetición ligera del paso de procesamiento, para no dejar acumular vacantes hasta la siguiente semana si el Lunes no alcanzó a cubrir todo el backlog.

8.5 VIERNES — Analytics

`~/vantage_pipeline.sh analytics`

Output: efectividad por fuente, tasa de links muertos por tipo de URL, ratio career pages vs. aggregators.

Acción concreta: si career pages producen menos de 5 resultados relevantes en la semana, ajusta el Prompt A (ver §13 — Prompts & Wrappers) — no el threshold de Score. (Recuerda §3, Filosofía de Fallo: el Score bajo no es el problema a corregir, el input de búsqueda sí lo es.)

Con esto se cierra el ciclo semanal. La siguiente vez que abras Claude para trabajar en VANTAGE, el ciclo completo empieza de

nuevo desde §6 — Ciclo de Sesión.

9. VANTAGE RUNTIME (Consulta Operativa) · ID: MANUAL:VANTAGE-RUNTIME-001

Ya viste varios de estos comandos en acción durante el flujo semanal (§8) — esta sección los reúne como catálogo de referencia completo, junto con el detalle de cuándo y por qué correr cada uno.

9.1 ¿Qué es el Runtime?

Es la herramienta de observabilidad del sistema. Permite interrogar a Notion y extraer contexto semántico sin salir de la terminal.

9.2 Comandos Principales — Mantenimiento del Tracker

Estos comandos operan sobre el estado del Tracker y están disponibles como subcomandos de vl1. Cada uno tiene un alcance preciso y un modo de operación por defecto.

- vl1 tracker — genera un reporte de estado del Tracker en tiempo real: distribución por Gate_Decision, conteo de entradas activas (CREATE + APPLIED), entradas BLOCKED, aplicaciones de los últimos 7 días y NADs vencidas. Es el punto de partida del ciclo semanal — corre antes de cualquier otra operación para tener visibilidad del estado actual (esto es lo que produce el output que viste en el Test Inicial de Setup, §4, Paso 5).
 - vl1 analytics — analiza la efectividad de las fuentes de discovery: qué canales producen más entradas CREATE, qué ratio de URLs funcionales tienen, cuál es el score promedio por fuente, y qué método de búsqueda (SEARCH-WEEK, SEARCH-EXEC, Manual) tiene mayor tasa de éxito. Corre los viernes como parte del cierre semanal (§8.5).
 - vl1 batch — modo de operación por defecto: read-only. Muestra la distribución por Status y el conteo de entradas que serían afectadas por la operación batch configurada en el script. Para ejecutar escritura, pasar el flag -execute explícitamente:

```
vl1 batch --execute
```

Sin --execute, el comando nunca escribe en Notion. Esta protección es permanente — no se puede desactivar sin modificar el flag.

- vl1 recovery — verifica la consistencia de los datos en el Tracker: detecta entradas sin Score, sin VM_Scope o sin Gate_Decision. También gestiona checkpoints del pipeline — si un run anterior falló a mitad, recovery carga el último checkpoint y permite retomar desde el paso fallado. Corre cuando el pipeline reporta inconsistencias o tras un fallo inesperado.
 - vl1 profile — gestiona la configuración del perfil activo del sistema: keywords VM y de pivote, pesos de scoring, empresas target por tier y foco geográfico. Permite actualizar el perfil sin editar código — los cambios se persisten en config/profile_config.yaml. Opción 7 ("Salir sin

cambios”) es el exit seguro; cualquier cambio guardado requiere propagación manual a `layer_1_run.py`.

- `v11 backfill` — backfill de campos Class A faltantes en entradas existentes: `layer`, `hash` y `Prioridad`. Opera con preview obligatorio antes de escribir — muestra exactamente qué entradas serán modificadas y por qué razón se infirió el layer. Acepta `-dry-run` para preview sin confirmación:

```
v11 backfill --dry-run
```

Sin `--dry-run`, solicita confirmación explícita (s) antes de cualquier escritura.

9.3 Cuándo correr sync

Correr `vantage.py sync` después de:

- Cualquier ciclo L1/L2 que haya escrito entradas nuevas en Notion.
 - Después de resolver entradas `REVIEW_NEEDED` en el Tracker (ver §8.2).
 - Si status muestra `"warning": "entity_index_stale"` (`index > 24h`).
 - Si status muestra `orphan_candidates > 0` de forma persistente.
 - No es necesario para cambios de `Status`, `Score`, `Gate_Decision` en páginas individuales — esos se leen en vivo vía `resolve/context`.

9.4 Runtime Build — Cuándo y Para Qué

El Runtime Build regenera los tres artefactos de lectura del sistema:

`entity_index_v2.json`, `graph_v2.json` y `backlinks_v2.json`. Se corre desde `Layer_1/scripts/` con el venv activo.

Cuándo correrlo:

- Después de cualquier migración de namespaces o cambio en `resolver_registry_v2.json`.
 - Si `graph_v2.json` muestra self-loops inesperados (síntoma de colisión de namespace).
 - Si `entity_index_v2.json` contiene IDs con prefix incorrecto.
 - Como parte del cierre formal de un release que afecte la capa de Runtime.
- El Build es determinista: el mismo Registry + el mismo estado de Notion producen los mismos artefactos. Si el resultado varía entre runs sin cambios en los inputs, es una señal de problema en el Registry — no en el Build.
- Sobre `resolver_registry_v2.json`: desde v2.4.0 (Runtime Contract Migration), este archivo es la fuente enforced — no solo declarada — de namespace ownership. Cada tipo de entidad tiene su `entity_prefix` definido aquí; ningún componente del sistema puede hardcodear ni inferir un prefix. Una edición manual que asigne un prefix incorrecto producirá colisiones de namespace en el siguiente Runtime Build, lo que se manifestará como self-loops en `graph_v2.json`. Antes de editar: verificar el prefix activo por tipo de entidad y correr Runtime Build para confirmar que no hay colisiones.

- Comandos relacionados de deduplicación y oportunidades:
`cd $LAYER_1_DIR && source .venv/bin/activate && python3 scripts/consolidate_duplicates.py # alias: vdedup`
`cd $LAYER_1_DIR && source .venv/bin/activate && python3 scripts/dedup_opportunities.py # alias: vopport`

——10. GESTIÓN DE DATOS · ID: MANUAL:GESTION-DATOS-001

Esta sección consolida en un solo lugar todo lo relacionado con exclusiones y deduplicación de vacantes — conceptos que se mencionan a lo largo de §1, §2 y §8, y que aquí tienen su definición completa y única.

Hard Blocks — Empresas Excluidas Permanentemente

Estas empresas o tipos de rol nunca entrarán al sistema. Se filtran en el origen (antes de que la vacante exista como registro en Notion) y no son recuperables bajo ninguna circunstancia, ni siquiera vía Dashboard:

- L'Oréal (todas las divisiones)
 - Levi Strauss & Co. (Levi's, Dockers)
 - El Palacio de Hierro
 - Roles store-level sin gestión estratégica

Soft Blocks — Bloqueos Recuperables

A diferencia de los Hard Blocks, estas vacantes sí pueden recuperarse: fueron bloqueadas por inconsistencias en datos Class A (URL rota, JD parcial) o por score insuficiente, no por pertenecer a una empresa vetada. Se recuperan corrigiendo el input incorrecto a través del Dashboard — el procedimiento completo está en §8.2 (Martes).

Deduplicación

- Ventana: 30 días. Una vacante que ya existe en el Tracker no se vuelve a crear si aparece de nuevo dentro de esta ventana.
 - Clave compuesta: brand + title + location.
 - Jerarquía entre capas: L1 > L2 > L3. Cuando dos capas detectan la misma vacante, persiste la instancia de la capa de mayor jerarquía, pero se toman de la capa de menor jerarquía los datos que puedan complementar sus propiedades Class A (esto es exactamente lo que ocurre en el paso de Consolidation & Dedup del Lunes, §8.1).

——11. HEALTH CHECK · ID: MANUAL:HEALTHCHECK-001

Red Flags — Ajustar Inputs, No Sistema

- Ready-to-Apply vacío por más de 3 días → ajustar Prompt A (ver §13 — Prompts & Wrappers), no el threshold. (Ver también §3 — Filosofía de Fallo.)
 - Career pages con éxito < 50% → revisar fuentes de discovery.
 - Pipeline runtime > 5 min → archivar entradas inactivas.

Qué es y qué lee

Es un script de arranque, de lectura estricta (cero escritura salvo la excepción documentada abajo). Corre automáticamente al invocar el alias start (activa venv +

carga env + ejecuta el script). También puede correrse manualmente:

```
cd Layer_1/scripts && python3 health_check.py
```

Qué lee, en este orden:

1. Versión del sistema — propiedad Versión de V-CHANGELOG vía Notion.
 2. Entorno (.env) — verifica que NOTION_TOKEN y demás vars requeridas existan.
 3. Git — git status --porcelain; reporta si hay archivos sin commitear.
 4. Último commit (vgit) — git log -1 para timestamp de referencia.
 5. Notion reachable — fetch mínimo a V-SYSTEM-PROMPT para confirmar conectividad y token válido.
 6. Docs fundacionales — confirma que los 6 documentos existen localmente en ACTIVE/.
 7. Último vdoc sync — cuál de los 6 docs locales tiene el mtime más reciente, y hace cuánto.
 8. Antigüedad de índices (index_age) — ver detalle abajo. Única sección con capacidad de escritura (auto-sync condicional).
 9. Tickets pendientes — Bug Tracker y Task Tracker, agrupados por prioridad.
10. Índices monitoreados: graph_v2.json y entity_index_v2.json, ambos en Layer_1/scripts/.
11. Comportamiento del auto-sync (desde v8.7.9): si algún índice supera 24 horas sin actualizarse, health_check.py dispara automáticamente python3 vantage.py sync (housekeeping de rutina — no requiere aprobación del operador, no es remediación de un fallo, según la misma lógica de §3 — Filosofía de Fallo: esto no es un error, es mantenimiento normal). El sync se dispara una sola vez por corrida, solo si al menos un índice cruzó el umbral — no re-sincroniza índices ya frescos, y no corre si todos están dentro del umbral.
12. Cómo leer el output:
 - ✓ verde — check pasó.
 - ! amarillo — advertencia, no bloquea (ej. índice stale antes del auto-sync, tickets pendientes).
 - X rojo — fallo real, contribuye al exit code final.
 - Línea final Sistema OK (exit 0) vs. Sistema con issues: [lista] (exit 1).
 - Si el auto-sync falla: aparece X index — auto-sync falló o auto-sync timeout. Esto sí es un fallo real — el script no reintenta. Acción manual: python3 vantage.py sync

desde Layer_1/scripts, y verificar con vantage.py status que entities_after >= entities_before.

Tickets pendientes: se listan explícitamente solo CRÍTICO y ALTO; MEDIO/BAJO/

Sin Prioridad aparecen solo como conteo — ver Notion para detalle.

Sync manual sigue disponible para forzar fuera de umbral, o si has realizado cambios masivos en el Tracker o el Canon y no quieres esperar a la siguiente corrida de start:

```
python3 vantage.py sync
```

El V-ID-Census

Qué es: el V-ID-CENSUS es tu mapa de navegación — te dice en qué documento y en qué bloque exacto vive cada ID del sistema, con link directo. Pero es un mapa, no el territorio: si el Kernel cambia y el Census no se actualiza, el mapa miente.

Este es el mismo Census que se verifica y actualiza durante el Ciclo de Sesión (§6).

Cuándo se regenera (obligatorio, no opcional):

- Antes de marcar cualquier ticket como cerrado, si ese ticket cambió el estado de un ID (de pendiente a resuelto, o creó uno nuevo).
 - Si no tienes Terminal a la mano en ese momento, el ticket se queda en Blocked-Census — no se cierra en falso, se marca como bloqueado hasta que puedas correr el script.
 - Cómo corre:

```
source ~/Documents/03 Projects/VANTAGE/Layer_1/.venv/bin/activate
cd ~/Documents/03 Projects/VANTAGE/Layer_1/scripts
python3 generate_census.py
```

El script también detecta IDs que existen en los documentos pero no en su lista de seguimiento ("huérfanos") y te los señala — ya no se cuelan en silencio. Y para cada ID resuelto genera el link exacto al bloque en Notion, no solo al documento.

Orden con Changelog: primero Census actualizado, después la entrada de Changelog. Nunca al revés (esto es exactamente el paso 2 del Cierre de Sesión, §6).

Al cerrar sesión: si hubo cambios a documentación o bases de datos, se te presenta automáticamente un resumen de lo que quedó hecho vs. pendiente — sin que tengas que pedirlo.

Aviso en arranque: health_check.py (alias start) reporta la antigüedad del Census en cada corrida — ! census — Nd sin regenerar si pasó el umbral de 7 días. Es solo un recordatorio visual, no dispara nada automáticamente; sigue siendo tu responsabilidad correr generate_census.py cuando cierres un ticket que cambió estado de un ID.

12. TROUBLESHOOTING · ID: MANUAL:TROUBLESHOOTING-001

Problemas Comunes y Soluciones

Pipeline no corre:

- Verificar .env en ~/vantage_notion_audit/.
 - Confirmar token Notion no expirado (regenerar en Notion → Settings → API → New token).
 - Verificar entorno Python activo: source Layer_1/.venv/bin/activate && python --version.
 - Confirmar permisos de ejecución: ls -la ~/vantage_pipeline.sh (debe tener x).
 - Entity Index desactualizado:
 - Desde v8.7.6: health_check.py detecta índices >24h y dispara vantage.py sync automáticamente en cada corrida de start — no requiere acción manual en el flujo normal.
 - Síntoma de que el auto-sync falló: health_check.py reporta × index — auto-sync falló o auto-sync timeout en vez del ✓ esperado.
 - Solución manual (solo si el auto-sync falló): python vantage.py sync desde Layer_1/scripts.
 - Verificar resultado: vantage.py status debe mostrar entities_after >= entities_before.
 - Si persiste: verificar token Notion y conectividad a internet.
- L3 no procesa correos:
 - Verificar layer_3.env existe en Layer_3/config/.
 - Confirmar credenciales: IMAP (Gmail), GROQ_API_KEY.
 - Ejecutar manualmente: vl3 (debe procesar hasta 5 correos).
 - Revisar heartbeat: cat ~/.vantage/l3_heartbeat.json (última ejecución exitosa).
 - Si falla autenticación IMAP: regenerar app password de Gmail.
- Figma plugin no resuelve IDs:
 - Verificar registry_seed.json actualizado desde lienzo Figma.
 - Confirmar que code.js tiene Registry V2 embebido (variable REGISTRY al inicio).
 - Comparar IDs en .md generado por CV-B vs IDs reales en capas Figma.
 - Si hay mismatch: regenerar registry_seed.json desde Developer Console de Figma.
- Reinstalar plugin si persiste: Plugins → Development → Import plugin from manifest.
- Dashboard no abre:
 - Verificar Flask corriendo: lsof -i :8000 (debe mostrar proceso Python).
 - Ejecutar smoke test: vd debe imprimir "SMOKE PASSED — abriendo dashboard".
 - Si falla smoke test: revisar dashboard_start.sh permisos (chmod +x).

- Verificar puerto 8000 libre: `killall -9 Python` si hay proceso zombie.
- Si error de importación: confirmar `.venv` activo y dependencias instaladas.
- `REVIEW_NEEDED` no se resuelve tras corregir:
- Confirmar que cambiaste Status → Target en Notion (no otro valor).
- Verificar que corrección se guardó (refrescar página Notion).
- Ejecutar pipeline: `~/vantage_pipeline.sh`.
- Si persiste: verificar en terminal qué campo sigue bloqueando (Python imprime razón).
- Revisar logs en `~/vantage/logs/` para diagnóstico detallado.
- `vl1 batch` modifica entradas sin `--execute`:
- Bug crítico: reportar inmediatamente.
- Workaround: verificar siempre con `vl1 batch` (sin flag) antes de ejecutar.
- Confirmar que script tiene guard if not args.execute: return al inicio.
- `vsync_doc.py` falla con error "`blocks.children.list()` returned None":
- Bug conocido de `notion-client 3.x`.
- Solución: `vsync_doc.py` usa `safe_list()` con `httpx` directo (3 reintentos).
- Si persiste: verificar que `page_id` sea válido y token tenga permisos de lectura.
- Alternativa temporal: sync manual vía MCP Notion.
- Score = 0 en vacante que parece relevante:
- Verificar que URL esté activa (no 404/403).
- Confirmar que JD contenga keywords VM (Python busca términos específicos).
- Revisar `VM_Scope` asignado (debe ser Core/Adjacent, no Off-Target).
- Si todo está correcto: revisar pesos de scoring en `profile_config.yaml`.
- No modificar Score manualmente (campo Class B, Python lo recalcula).
- Gate = BLOCKED recuperable pero el Dashboard no lo detecta:
- Confirmar que entrada aparece en dropdown del Dashboard.
- Verificar que `Gate_Decision` = BLOCKED (no EXPIRED ni vacío).
- Si no aparece: refrescar cache de Runtime (`vantage.py sync`).
- Si aparece pero validación falla: revisar logs de `run_pipeline.py` en Dashboard.

Referencias a documentación adicional

- Filosofía de fallo: `KERNEL:FAIL-PHILOSOPHY` (ver también §3 de este Manual).
- Reglas de Oro: `KERNEL:CV-GOLDEN-RULES` (ver también §16 de este Manual).
- Schema de datos: `KERNEL:SCHEMA`.
- Gate Decisions: `KERNEL:GATE-DECISION` (ver también §2 de este Manual).

Se consultan vía MCP desde la PROMPT LIBRARY en Notion. Este es el catálogo que Perplexity Desktop lee cada Lunes (§8.1) para ensamblar los prompts de L1 y L2: los Prompt Bases (BASE SPEC L1, BASE SPEC L2) y los Wrappers correspondientes (Career Sites, LinkedIn, Aggregators, Gemini, Grok, you.com, Prompt A/B/C, Prompt E de consolidación).

14. CHEAT SHEETS · ID: MANUAL:CHEATSHEETS-001

Cómo la IA lee el KERNEL y el CAREER CANON (Lazy Load)

La extracción de reglas y contratos lógicos (Lazy Load) opera con la siguiente prioridad:

Prioridad A — Terminal (canónico): lazy_loader.py ejecuta Server-Side Lazy Load. Parsea bloques hijos de la Notion API y devuelve únicamente el payload del ID solicitado. Consumo: ~150 tokens por llamada.

Prioridad B — MCP Notion: reservado exclusivamente para escrituras (APROBAR_WRITE) y modificaciones estructurales de páginas. No se usa para lectura de reglas o contratos.

15. CRITERIO DE CALIDAD PARA PARCHES DOCUMENTALES · ID: MANUAL:PATCH-QUALITY-001

Todo parche a los 6 documentos fundacionales debe cumplir estos cinco criterios antes de aplicarse — si falla alguno, se reescribe antes de solicitar

APROBAR_WRITE:

1. Invisibilidad estructural — no crea secciones nuevas si el contenido cabe en una existente.
2. Continuidad de voz — mismo registro y nivel técnico del bloque que lo rodea.
3. Progresión narrativa intacta — el lector no debe notar un salto temático al leer de corrido.
4. Diff mínimo — se edita solo el texto indispensable, nunca el bloque completo si un párrafo basta.
5. Coherencia transversal — no puede contradecir ni duplicar una definición ya existente en Kernel, System Prompt, Career Canon o Aliases.
6. Un parche que pasa estos cinco filtros no se distingue, seis meses después, del texto que rodeaba su punto de inserción original.

——16. REGLAS DE ORO PARA OPERADORES · ID: MANUAL:REGLAS-DE-ORO-001

Base: KERNEL:CV-GOLDEN-RULES.

El contenido detallado de estas reglas vive en el Kernel del sistema y no está reproducido en este Manual más allá de esta referencia. Ver lista de huecos

detectados al final de este documento.

——17. SLA DE LATENCIA POST-INGESTA · ID: MANUAL:SLA-001

Nota: el SLA "< 45 minutos" cubre únicamente el segmento Score calculado → Ready-to-Apply (Discovery → Ready-to-Apply en nomenclatura anterior). El segmento Trigger → Score depende del ciclo de ejecución de ~/vantage_pipeline.sh (ver §8.1, Lunes) — no tiene SLA fijo salvo ejecución manual explícita de layer_1_run.py.

16. Reglas de Oro CV — Referencia Operativa Las Reglas de Oro (KERNEL:CV-GOLDEN-RULES) son restricciones de arquitectura, no preferencias. Viven íntegras en el Kernel — esta sección es un índice de navegación, no una copia.

ID	Regla	Qué bloquea
KERNEL:CV-GOLDEN-RULES-001	No Evaluar Fit Antes de Escribir	Preguntas de "¿me conviene esta vacante?" — el fit lo decide Score (Python) + el operador
KERNEL:CV-GOLDEN-RULES-002	No Calcular ni Estimar Campos Class B	Estimar Score, Gate_Decision, VM_Scope, etc. — son Python-only
KERNEL:CV-GOLDEN-RULES-003	No Cuestionar la Calidad de Datos del Usuario	Comentarios sobre volumen/calidad del JSON de búsqueda — estrategia es 100% humana
KERNEL:CV-GOLDEN-RULES-004	No Delegar Escritura al Usuario	"Copia esto y pégalo en Notion" — el sistema escribe directo, salvo export PDF/Drive
KERNEL:CV-GOLDEN-RULES-005	No Interpretar en SYNC	SYNC reporta datos puros, sin análisis ni recomendaciones
Toda violación produce el Template Universal de Rechazo (ver Kernel): OPERACIÓN RECHAZADA → razón → alternativa operativa → confirmación SÍ/ CANCELAR.		

Para el detalle completo de cada regla (ejemplos de solicitudes que la activan, redacción exacta de la respuesta estandarizada), consultar directamente KERNEL:CV-GOLDEN-RULES en el Kernel — fuente única, no se replica aquí para evitar drift entre documentos.		
--	--	--

17. Positioning Modes (N1–N4) — Criterio de

SelecciónCANON:POSITIONING-001 define 4 modos de posicionamiento para CV-B. Esta sección resuelve el gap operativo: con qué criterio elegir uno.

Modo	ID	Ancla canónica	Cuándo aplica
N1	CANON:POSITIO NING-N1	C01 · 3 marcas lujo · CAPEX/ OPEX · NPI	JD enfatiza gestión multi- marca de lujo, presupuesto, lanzamientos de producto
N2	CANON:POSITIO NING-N2	C02 · Adidas Brand Center · KPI07 · blueprints	JD enfatiza Store Design, Flagship, construcción/ remodelación física
N3	CANON:POSITIO NING-N3	C03 · 270+ POS · 6 países · KPI03– 06 · CF05	JD enfatiza rollout regional multi-país, estandarización, eficiencia operativa

N4	CANON:POSITIO NING-N4	C04/C05 · +43% tráfico · +18% conversión · 21 reportes	JD enfatiza liderazgo de campo comercial, KPIs de tráfico/ conversión, gestión de equipos directos
Regla de desempate (JDs híbridos) — ver CANON:POSITIO NING-001 para el texto completo: (1) más keywords mapeados al ancla, (2) empate → mayor seniority (N2>N1, N4>N3 con presupuesto regional explícito), (3) empate persistente → escalar a decisión humana vía fit_gaps.			

18. Golden Skeleton — Qué es y Dónde Vive

El "Golden Skeleton" (CANON:OUTPUT-CONTRACT-SKELETON-001) es la secuencia fija de bloques ##### figma_text_id que todo CV-B debe replicar exactamente — mismo conteo, mismo orden, solo cambia el contenido textual.

- SSOT de IDs de nodo Figma: registry_seed.json en 04-Vantage_CV/Figma Sync/.
 - Slots clave: 2055:9 (Nombre), 2055:10 (Tagline), 2043:51 (Perfil), 2043:56-60 (Skills), 2043:64+ (Experiencia).
 - Regla de invariancia: si el Skeleton cambia en Figma, registry_seed.json se actualiza antes del siguiente CV-B — nunca al revés.
 - Detalle completo del protocolo (immutability, slot integrity, null-fill rule) vive en CANON:OUTPUT-CONTRACT-001 — no se replica aquí.

19. Schema Class A/B — Referencia de Campos

KERNEL:SCHEMA-001 define ownership exclusivo por campo. Esta tabla es índice de consulta rápida — el contrato completo (reglas de excepción, mapeo de vocabulario) vive en el Kernel.

Class A — Human-Primary (operador/feed_processor escriben):

Rol · Marca · Source_Type · URL · Status · Prioridad · Holding · JD · NAD · layer · hash

Class B — System-Primary (Python únicamente, ningún otro componente escribe):

Score · Gate_Decision · VM_Scope · Role_Class · Match · Next_Action · Fetch · Fuente

Excepción documentada: Fuente_Manual (Class A) existe para valores de fuente que deben persistir entre runs — Fuente (Class B) se sobrescribe en cada corrida (KERNEL:SCHEMA-003).

Pesos de Score/VM_Scope: viven en profile_config.yaml, propiedad de Python — el Manual no reproduce los valores numéricos porque son deuda de implementación, no contrato documental (ver KERNEL:GATE-DECISION-002). Un operador que necesite ajustar pesos debe editar ese archivo directamente, no este documento.